5-2 Milestone Four: Enhancement Three: Databases

Andrew Allen

Department of Computer Science, Southern New Hampshire University

CS 499: Computer Science Capstone

Dr. Sherin Aly

August 2, 2026

Enhancement Three: Databases

Chosen Artifact

The artifact I chose for inclusion in my ePortfolio for this enhancement is an Android Inventory Tracking application I originally created during my CS360 Mobile Architect and Programming course. This application allows users to create, view, update, and delete inventory items while tracking item information and quantities. The original version was created with Java and Android Studio and provided the main functionality needed to manage inventory data locally. For this enhancement I implemented Room, Androids object-relational mapping (ORM) library, to provide a safer and more structured interface between the app and its SQLite database.

The enhanced database architecture includes an AppDatabase class, an InventoryDao data access object, an inventory repository, and database unit tests. These components provide a clear separation between the apps UI, logic, and data persistence layers. The application uses the Model-View-ViewModel (MVVM) pattern, allowing the ViewModel to communicate with the repository rather than directly accessing the database like the original app. This structure supports asynchronous database operations and makes the app easier to test and expand.

Justification for Inclusion

I selected this artifact for my ePortfolio because it demonstrates my ability to design, implement, test, and maintain a structured database solution. This artifact demonstrates how database concepts can be applied to solve a practical problem by allowing users to maintain accurate inventory records through a persistent local database.

Many components of this enhancement demonstrate my database skills. InventoryDao for example, defines the database operations used by the app, including inserting, retrieving,

updating, and deleting records. By using a data access object, database operations are organized into a dedicated layer rather than being distributed throughout the app. This improves separation of concerns and makes the database code easier to understand, test, and maintain.

The Room database implementation also provides a structured mapping between Kotlin data classes and SQLite database tables. Room reduces the amount of repetitive database code required by handling a lot of communication between the app and SQLite for us. It also provides compile-time validation of queries, which helps identify SQL errors during development. This improved the reliability and maintainability of the apps data layer.

The repository pattern was incorporated to create an additional layer of abstraction between the ViewModel and database. The repository manages access to inventory data and prevents high level app components form depending directly on the database implementation. This design encourages codebase modularity and makes it easier to modify and expand later. The app also uses Kotlin coroutines and asynchronous data streams to perform database operations outside the main UI thread. This keeps the app snappy and smooth while inventory items are being modified or retrieved.

Another improvement performed during this enhancement was the creation of database unit tests for InventoryDao. The tests verify that essential database operations function correctly, such as inserting new items, retrieving stored records, updating existing records, and deleting records. Testing the data access layer helped confirm that the app could reliably store and manage inventory information without physically testing on a real device. This also emphasized the importance of validating database behavior independently from the UI.

Overall, this enhancement replaced a less structured approach to local data management with modern Room ORM architecture. This improved implementation provides stronger separation of concerns, compile-time query validation, and automated testing. These changes make the app far more consistent with Android development best practices and improve the quality and reliability of its database component.

Course Outcome Coverage

The enhancements completed for this artifact successfully meet the databases outcomes I identified originally for this enhancement, in particular, course outcomes four and five.

For course outcome four: Demonstrate an ability to use well-founded and innovative techniques, skills, and tools in computing practices for the purpose of implementing computer solutions that deliver value and accomplish industry-specific goals. The implementation of Room, SQLite, data access objects, repositories, Kotlin coroutines, and database unit tests demonstrates the use of established technologies and professional development practices to create a reliable data persistent solution. This enhanced database architecture supports the app's primary purpose of allowing inventory information to be stored, retrieved, updated, and deleted consistently. This provides practical value by allowing users to manage records that remain available after the app has been closed or restarted.

Now, for course outcome five: Develop a security mindset that anticipates adversarial exploits in software architecture and designs to expose potential vulnerabilities, mitigate design flaws, and ensure privacy and enhanced security of data and resources. Even though the inventory app does not currently store highly sensitive personal information, I applied a security mindset by considering how the database architecture could reduce potential risks to data

integrity and reliability. Rather than allowing different parts of the app to interact directly with the database, I implemented a structured data layer using Room, a dedicated data access object, and a repository. This design limits database operations to defined methods and creates clear boundaries between the UI, logic, and persistent layers.

Room also provides compile-time validation of database queries, which helps identify query-related errors during development before they cause unexpected errors or behavior when using the app. Also, Kotlin coroutines were used to perform database operations asynchronously rather than on the main apps thread. This improved performance and ensured that database activity is handled separately.

The addition of InventoryDaoTest further supports my security and reliability mindset by verifying that database operations behaved as expected. The tests evaluated the essential operations of inserting, retrieving, updating, and deleting records. Testing these functions helps identify potential defects that could affect the accuracy and integrity of stored records. It also provides a repeatable method for detecting regressions in the future when I add more features to the app.

Reflection

Enhancing the database component gave me a much stronger practical understanding of how modern Android apps organize, isolate, and manage persistent data. Before completing this enhancement, I only understood basic SQL operations. I have now learned how to integrate complete multi-layered MVVM database solutions. Implementing Room taught me how the ORM library can simplify communication between an app and its database while retaining performance with SQLite.

One of the most valuable lessons I learned was the importance of maintaining a clear separation of responsibilities across app layers. Isolating the database operations with InventoryDao, managing the data access through RoomInventoryRepository, and keeping database logic separate from the UI layer made the codebase easier to read and test. I learned that effective database engineering involves a lot more than just creating tables and writing queries. It also required careful consideration of data flow, app responsiveness, concurrency, data integrity, and maintainability of the codebase over time.

A challenge I faced during this enhancement was managing asynchronous data streams and thread boundaries. Database operations should not run on the main UI thread because it can slow or stop the UI from responding. Integrating Kotlin Coroutines and asynchronous data streams required careful attention to how inventory data moved from the database through the repository and ViewModel before reaching the UI. Resolving these challenges strengthened my understanding of background processing and taught me the importance of multithreading.

Another challenge I encountered was configuring isolated database unit tests in InventoryDaoTest. Establishing an in-memory database instance allowed me to execute and verify instert, retrieve, update, and delete operations independently of the UI. Creating these tests verified the importance of automated testing and I learned a way to verify database operations without relying on UI visual feedback. These tests also provide me with a way to test for regressions in the future as I continue to expand the app.

Overall, this enhancement has strengthened my skills in database design, asynchronous programming, software architecture, and automated testing. It also improved my understanding of how database design decisions affect the overall performance and integrity of the entire app.